

CSE115L Project: Academic Transcript Generator in C

Objective

The goal of this project is to allow generation of customized and well-designed academic transcripts by performing necessary calculations and analysis using user-input data, and then printing them in a well-formatted manner in the terminal/console.

Tools Used

- **C Compiler:** GNU C Compiler (GCC) version 16.1.1
- **Build Tool:**
 - GNU Make on GNU/Linux and Unix-like systems
 - Manual on Windows
- **Editor/IDE:**
 - Emacs on GNU/Linux
 - VSCode on Darwin (macOS)
 - Code::Blocks on Windows

Features

- **Support for multiple semesters:** Allows inclusion of multi-semester data without limits.¹
- **Robust data analysis:** On-the-fly TGPA and CGPA calculation.
- **Identity processing:** Allows the user to enter basic profile data such as their name.
- **Command-line interface (CLI):** User-friendly interface and prompt.
- **C99 standard:** Codebase follows GNU C99 standard for maximum compatibility.

1. There is no practical limit in the implementation; however, this does not stop other limits from existing. Physical, hardware and operating system limits still apply.

Example Runs

The following example command was executed on a Windows 11 x86_64 machine, using the Code::Blocks IDE, which invokes gcc (the compiler) and the compiled program atsgen.

```
Starting interactive transcript generation.
Enter your name: Ar Rakin
Enter semester number (e.g. 261; enter 0 to stop): 253
Courses are listed below, please enter the appropriate course IDs indicated inside square brackets '[]'.
**Enter 0 to stop adding courses and finalize the semester data.**

[1]: CSE115 - Programming Language I
[2]: CSE115L - Programming Language I Lab
[3]: CSE173 - Discrete Mathematics
[4]: CSE215 - Programming Language II
[5]: CSE215L - Programming Language II Lab
[6]: CSE225 - Data Structures & Algorithms
[7]: CSE225L - Data Structures & Algorithms Lab
[8]: MAT116 - Pre-calculus
[9]: MAT120 - Calculus I
[10]: MAT125 - Linear Algebra & Analytical Geometry
[11]: ENG103 - Intermediate Composition
[12]: ENG105 - Advanced Composition
```

[13]: ENG111 - Public Speaking

Enter course ID to add: 1

Enter marks for [1] CSE115 - Programming Language I: 94

Course added: CSE115 - Programming Language I [Marks: 94/100]

Enter course ID to add: 2

Enter marks for [2] CSE115L - Programming Language I Lab: 100

Course added: CSE115L - Programming Language I Lab [Marks: 100/100]

Enter course ID to add: 8

Enter marks for [8] MAT116 - Pre-calculus: 94

Course added: MAT116 - Pre-calculus [Marks: 94/100]

Enter course ID to add: 11

Enter marks for [11] ENG103 - Intermediate Composition: 89

Course added: ENG103 - Intermediate Composition [Marks: 89/100]

Enter course ID to add: 0

Enter semester number (e.g. 261; enter 0 to stop): 261

Courses are listed below, please enter the appropriate course IDs indicated inside square brackets '[''].
Enter 0 to stop adding courses and finalize the semester data.

[1]: CSE115 - Programming Language I

[2]: CSE115L - Programming Language I Lab

[3]: CSE173 - Discrete Mathematics

[4]: CSE215 - Programming Language II

[5]: CSE215L - Programming Language II Lab

[6]: CSE225 - Data Structures & Algorithms

[7]: CSE225L - Data Structures & Algorithms Lab

[8]: MAT116 - Pre-calculus

[9]: MAT120 - Calculus I

[10]: MAT125 - Linear Algebra & Analytical Geometry

[11]: ENG103 - Intermediate Composition

[12]: ENG105 - Advanced Composition

[13]: ENG111 - Public Speaking

Enter course ID to add: 3

Enter marks for [3] CSE173 - Discrete Mathematics: 99

Course added: CSE173 - Discrete Mathematics [Marks: 99/100]

Enter course ID to add: 9

Enter marks for [9] MAT120 - Calculus I: 94

Course added: MAT120 - Calculus I [Marks: 94/100]

Enter course ID to add: 10

Enter marks for [10] MAT125 - Linear Algebra & Analytical Geometry: 99

Course added: MAT125 - Linear Algebra & Analytical Geometry [Marks: 99/100]

Enter course ID to add: 13

Enter marks for [13] ENG111 - Public Speaking: 90

Course added: ENG111 - Public Speaking [Marks: 90/100]

Enter course ID to add: 0

Enter semester number (e.g. 261; enter 0 to stop): 0

=====

Name:	Ar Rakin
-------	----------

=====

Semester:	253
-----------	-----

ID	Course	Credits	Marks	GP	Grade
[1]	CSE115 - Programming Language I	3	94	4.00	A
[2]	CSE115L - Programming Language I Lab	1	100	4.00	A
[8]	MAT116 - Pre-calculus	3	94	4.00	A
[11]	ENG103 - Intermediate Composition	3	89	3.30	B+

TGPA: 3.79 (A-)

CGPA: 3.79 (A-)

=====

Semester:	261
-----------	-----

ID	Course	Credits	Marks	GP	Grade
[3]	CSE173 - Discrete Mathematics	3	99	4.00	A
[9]	MAT120 - Calculus I	3	94	4.00	A
[10]	MAT125 - Linear Algebra & Analytical Geometry	3	99	4.00	A
[13]	ENG111 - Public Speaking	3	90	3.70	A-

```
TGPA:          3.93 (A-)
CGPA:          3.86 (A-)
=====
Final CGPA:    3.86 (A-)
=====
**** End of transcript ****

Process completed with code 0. Press any key to continue...
```

Platform-specific Build Instructions

Windows

First, open the `atsgen.c` file in Code::Blocks. Then, to run the code: Build > Build & Run. This should build and run the project successfully.

GNU/Linux & Unix-like systems

To run the code on a GNU/Linux or any Unix-like machine, the `make` utility can be used. For example, on typical GNU/Linux systems, the following command:

```
$ make run-linux
```

Will compile and execute the program.

Significant Code Snippets

The following functions (only signatures are given) are the backbone of the program:

❑ `void print_transcript(char *name, struct semester_record_list *record_list);`

This function prints the transcript data to the terminal/console. It handles formatting, decimal rounding, string processing, etc.

❑ `double calc_gp(int marks);`

A helper function to calculate grade points from the given marks in 0-100 range.

❑ `char *get_letter_grade(double gp);`

A helper function to determine the letter grade from the given grade point, according common U.S. university grading systems.

❑ `void add_courses_interactive(int added_courses[],
 int added_course_marks[],
 int *added_course_count);`

Interactively takes course information input from user, and stores them in memory.

Contribution

Contributors are listed in alphabetical order.

Contributor	Contribution
Akib Elahi	-
Ar Rakin	Reviewed the entire code, reformatted, refactored and restructured the codebase, fixed unnoticed bugs and improved compatibility – 40%
Momsad Hossain	-

Conclusion

This project aims to successfully demonstrate the core knowledge and application of C programming concepts into a real-world usable tool. It has some areas for improvement, such as integrating a remote database or handling rare edge cases.